

TECHNICAL REPORT

Penetration Test Report

Lab Environment

Client / Scope:	<i>Hacksmarter.org</i>
Performed by:	<i>Leonardo Cardoso</i>
Date:	<i>07/05/26</i>
Classification:	Confidential

Table of Contents

Table of Contents	2
1. Introduction.....	4
2. Scope and Methodology	4
3. Risk Classification	4
4. Summary of Identified Vulnerabilities	5
5. Vulnerability Details.....	6
5.1. SQL Injection (UNION-based).....	6
Description.....	6
Impact	6
Recommendation	6
Evidence.....	6
5.2. Credential Exposure via SQL Injection.....	7
Description.....	7
Impact	7
Recommendation	7
Evidence.....	8
5.3. Account Registration Without Security Controls	8
Description.....	8
Impact	8
Recommendation	8
Evidence.....	9
Evidence.....	9
5.4. Local File Inclusion (LFI).....	10
Description.....	10
Impact	10
Recommendation	10
Evidence.....	10
5.5. Cross-Site Scripting (XSS)	10
Description.....	10
Impact	10
Recommendation	10
Evidence.....	11

5.6. IDOR — Product Manipulation via ID	11
Description.....	11
Impact	11
Recommendation	11
Evidence.....	11
5.7. Exposure of Hidden Products (Access Control Flaw)	12
Description.....	12
Impact	12
Recommendation	12
Evidence.....	12
5.8. Exposed Hidden Directories.....	13
Description.....	13
Impact	13
Recommendation	13
Evidence.....	13
5.9. Missing Clickjacking Protection (Iframe)	14
Description.....	14
Impact	14
Recommendation	14
Evidence.....	14
5.10. Improper Session Management (Cookie Not Invalidated)	15
Description.....	15
Impact	15
Recommendation	15
Evidence.....	15
6. Conclusion	15

1. Introduction

This document presents the results of the penetration test conducted in a lab environment, with the objective of identifying security vulnerabilities present in the assessed application, evaluating their associated risks, and proposing remediation recommendations.

The tests were conducted in a controlled manner, without the use of destructive techniques, focusing on identifying input validation flaws, access control issues, session configuration weaknesses, and improper exposure of information.

2. Scope and Methodology

The scope of this test covered the web application made available in the lab environment, including the authentication, user registration, and product catalog/purchase modules, as well as other exposed functionalities.

The methodology adopted followed web application penetration testing best practices (based on references such as the OWASP Testing Guide), comprising the following stages:

- Reconnaissance and mapping of the application and its directories/endpoints;
- Identification of data entry points (forms, URL parameters, cookies);
- Injection testing (SQL Injection, XSS);
- Access control and business logic testing (IDOR);
- Session management and authentication testing;
- Analysis of security headers and Clickjacking protection configurations;
- Documentation and risk classification of the identified vulnerabilities.

3. Risk Classification

Each identified vulnerability was classified according to its severity level, taking into account ease of exploitation and potential impact on the confidentiality, integrity, and availability of data:

CRITICAL — Trivial exploitation, severe and immediate impact on system data

HIGH — Feasible exploitation, relevant impact on data or functionality

MEDIUM — Exploitation possible under certain conditions, moderate impact

LOW — Difficult exploitation or limited impact

4. Summary of Identified Vulnerabilities

The table below consolidates the vulnerabilities identified during the test, individually detailed in section 5.

ID	Vulnerability	Severity
01	SQL Injection (UNION-based)	Critical
02	Credential Exposure via SQL Injection	Critical
03	Account Registration Without Security Controls	High
04	Local File Inclusion (LFI)	High
05	Cross-Site Scripting (XSS)	High
06	IDOR – Product Manipulation via ID	High
07	Exposure of Hidden Products (Access Control Flaw)	Medium
08	Exposed Hidden Directories	Medium
09	Missing Clickjacking Protection (Iframe)	Medium
10	Improper Session Management (Cookie Not Invalidated)	High

5. Vulnerability Details

5.1. SQL Injection (UNION-based)

SEVERITY: CRITICAL

Description

A parameter vulnerable to SQL Injection was identified in the application. Initially, the structure of the underlying SQL query was mapped using the incremental column-ordering technique, allowing the exact number of columns returned by the original query to be determined.

With the number of columns identified, it was possible to use the UNION-based SQL Injection technique to inject an additional query and extract information from the database, including the names of existing tables from the database's metadata schema (information_schema).

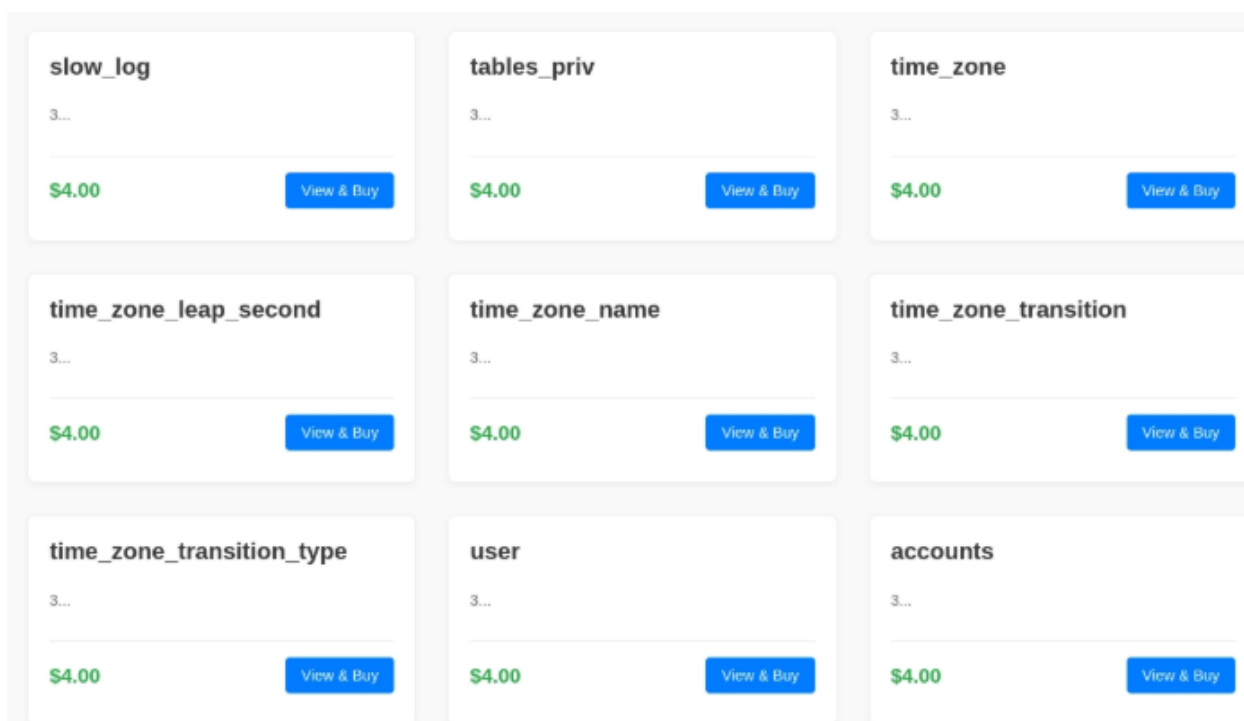
Impact

Exploiting this vulnerability allows an attacker to enumerate the entire database structure (tables and columns), serving as a foundation for more advanced attacks aimed at extracting sensitive data, including user credentials.

Recommendation

- Use parameterized queries (prepared statements) in all database interactions;
- Apply rigorous validation and sanitization of all user-supplied input;
- Implement a WAF (Web Application Firewall) as an additional layer of protection;
- Restrict the privileges of the database user used by the application (principle of least privilege);
- Disable detailed database error messages in the production environment.

Evidence



5.2. Credential Exposure via SQL Injection

SEVERITY: CRITICAL

Description

Building on the same SQL Injection vulnerability, it was possible to extract usernames and their corresponding password hashes directly from the users table stored in the database.

The obtained hashes were subsequently submitted to a password cracking process using a specialized tool, based on a public wordlist, which allowed the plaintext passwords of registered accounts in the application to be recovered.

Impact

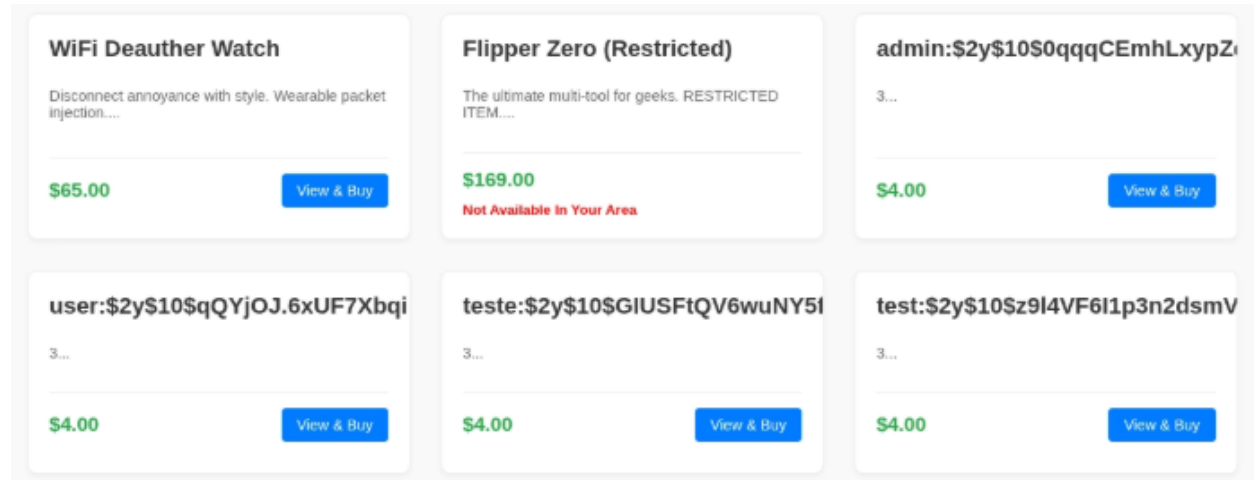
Obtaining valid user credentials fully compromises the confidentiality of the affected accounts, potentially allowing an attacker to log in as any user — including possible administrative accounts — as well as enabling password reuse attacks against other systems.

Recommendation

- Fix the SQL Injection vulnerability described in item 5.1, eliminating the root cause of the exposure;
- Store passwords using strong, salted hashing algorithms (e.g., bcrypt, Argon2), never fast, unsalted hash formats;

- Implement strong password policies and multi-factor authentication (MFA);
- Rotate/reset credentials for all potentially compromised users.

Evidence



`$2y$10$qQYjOJ.6xUF7XbqiF1DDBOC0mUKA88LtKSF8.D6HSvVU7wME5bb8u:password`

5.3. Account Registration Without Security Controls

SEVERITY: HIGH

Description

The account creation form does not enforce minimum security validations. The following flaws were identified:

Impact

The absence of these controls facilitates automated attacks, such as mass creation of fake accounts (feature abuse), enumeration of registered users, and the use of weak passwords that are easily broken through brute-force or dictionary attacks.

Recommendation

- Implement username validation (length, allowed characters, uniqueness without disclosing details to unauthenticated users);
- Standardize error messages so they do not reveal whether an email is already registered (avoiding user enumeration);
- Define a strong password policy requiring a minimum length and the use of special characters, numbers, and upper/lowercase letters;
- Implement CAPTCHA and rate limiting on the registration endpoint to prevent mass account creation;
- Add email confirmation (double opt-in) before fully activating the account.

5.4. Local File Inclusion (LFI)

SEVERITY: HIGH

Description

An application parameter vulnerable to Local File Inclusion was identified, allowing the inclusion and reading of local server files that should not be externally accessible.

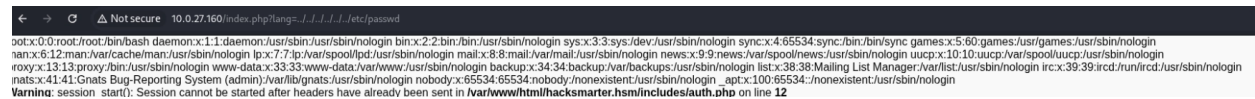
Impact

Exploiting this vulnerability can allow the reading of sensitive operating system and application files (configuration files, credentials, logs), and in more advanced scenarios could escalate to remote code execution (RCE), depending on the server configuration.

Recommendation

- Avoid using user-supplied input directly to build file paths on the system;
- Use an allowlist of valid files/directories;
- Restrict the application process's file system permissions (principle of least privilege);
- Disable dynamic remote file inclusion functions when not strictly necessary.

Evidence



5.5. Cross-Site Scripting (XSS)

SEVERITY: HIGH

Description

An entry point was identified in the application that does not properly sanitize/encode user-supplied data before reflecting it on the page, allowing the injection and execution of arbitrary JavaScript code within the victim's browser context.

The vulnerability was confirmed by executing a simple script capable of accessing the user's session cookie (document.cookie), which also revealed the absence of the HttpOnly flag on the session cookie.

Impact

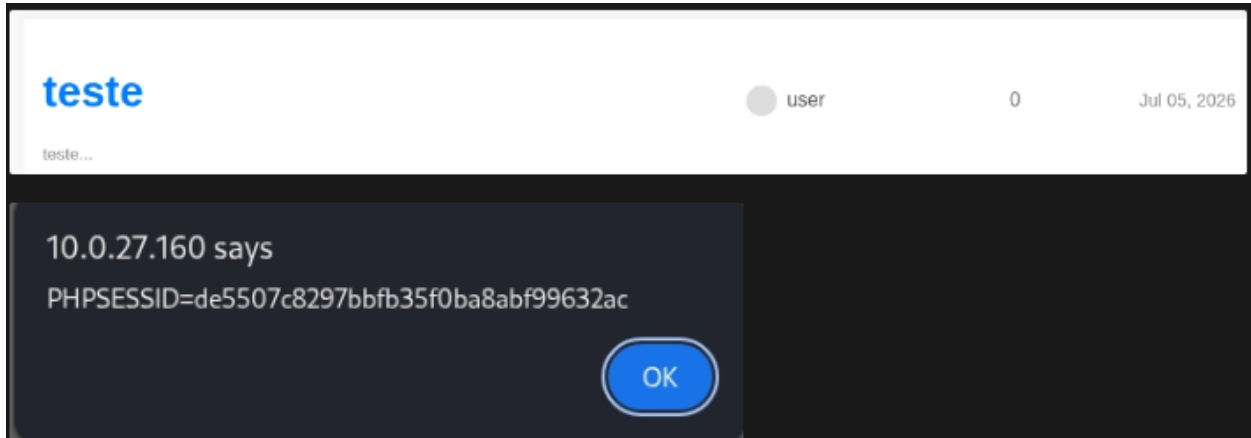
Exploiting XSS can allow the theft of session cookies and authentication tokens, session hijacking, redirection to malicious sites, phishing, and defacement of the application.

Recommendation

- Apply context-appropriate output encoding (HTML, attribute, JavaScript) to all content reflected on the page;

- Use well-established input sanitization libraries instead of custom filters;
- Implement the HttpOnly (and Secure) flag on session cookies, preventing access via JavaScript;
- Implement a restrictive Content Security Policy (CSP).

Evidence



5.6. IDOR — Product Manipulation via ID

SEVERITY: HIGH

Description

During the purchase flow, it was identified that the product identifier (ID) is sent and can be directly manipulated by the client, without proper server-side validation, allowing a user to change the selected product or its associated parameters during checkout.

Impact

This flaw constitutes an Insecure Direct Object Reference (IDOR) / business logic vulnerability, potentially allowing the manipulation of prices, quantities, or purchased products, causing direct financial loss to the business.

Recommendation

- Validate on the server, at every step of the purchase, whether the product and its attributes (price, availability) match the official data stored in the backend;
- Never trust price/product values sent by the client; always recalculate them on the server;
- Implement object-level authorization checks (verifying whether the user has permission over that specific resource).

Evidence

Your Shopping Cart

Product	Price	Qty	Total	Action
Encrypted USB Drive	\$85.00	1	\$85.00	Remove
Mechanical Keyboard	\$120.00	2	\$240.00	Remove
WiFi Deauther Watch	\$65.00	1	\$65.00	Remove
			Total: \$390.00	
			Continue Shopping	Proceed to Checkout →

5.7. Exposure of Hidden Products (Access Control Flaw)

SEVERITY: MEDIUM

Description

It was identified that it is possible to list and access products that are not configured to be displayed in the application's public panel/catalog, indicating a lack of proper access control at the object/resource level.

Impact

Information about unpublished products (which may include items under development, undisclosed promotions, or sensitive business data) may be improperly exposed to unauthorized users.

Recommendation

- Implement product status/visibility validation in the backend before returning any information to the client;
- Ensure that product listing/detail endpoints follow the same visibility rules applied to the administrative panel.

Evidence

ID	Payload 1	Status	Length ^	Round-trip Time (ms)	Request Sent At
1	0	200	3063	295	2026-07-05 12:36:02
9	8	200	3063	293	2026-07-05 12:36:02
8	2	200	4737	297	2026-07-05 12:36:02
2	1	200	5565	297	2026-07-05 12:36:02
10	9	200	5565	299	2026-07-05 12:36:02
4	3	200	6393	301	2026-07-05 12:36:02
5	4	200	7221	298	2026-07-05 12:36:02
6	5	200	8057	291	2026-07-05 12:36:03
8	7	200	8057	293	2026-07-05 12:36:03
7	6	200	8905	289	2026-07-05 12:36:03
11	10	200	8905	467	2026-07-05 12:36:03
12	11	200	8905	467	2026-07-05 12:36:03
13	12	200	8905	471	2026-07-05 12:36:03
14	13	200	8905	459	2026-07-05 12:36:03
15	14	200	8905	451	2026-07-05 12:36:03

31 requests

Request

```

1 POST /cart.php HTTP/1.1
2 Host: 10.8.27.169
3 Content-Length: 23
4 Cache-Control: max-age=0
5 Origin: http://10.8.27.169
6 Content-Type: application/x-www-form-urlencoded
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/145.0.0.0 Safari/537.36
9 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
10 Referer: http://10.8.27.169/product.php?id=1
11 Accept-Encoding: gzip, deflate
12 Accept-Language: en-US,en;q=0.9
13 Cookie: PHPSESSID=de5507c8297bbfb35f0ba8abf99632ac
14 Connection: close
15 action=add&product_id=6

```

Response

HackSmarter

Search products

Home Store Blog Community Forum Profile (user) Logout

Your Shopping Cart

Product	Price	Qty	Total	Action
Mechanical Keyboard	\$120.00	1	\$120.00	Remove
Hack Smarter Hoodie	\$49.99	1	\$49.99	Remove
Encrypted USB Drive	\$85.00	1	\$85.00	Remove
WiFi Deauther Watch	\$65.00	1	\$65.00	Remove
Flipper Zero (Restricted)	\$169.00	1	\$169.00	Remove
Quantum Decryptor (Unreleased)	\$9,999.00	1	\$9,999.00	Remove
			Total: \$10,487.99	

5.8. Exposed Hidden Directories

SEVERITY: MEDIUM

Description

Using a directory scanning tool, hidden directories and/or files were identified in the application that are not publicly referenced but were nonetheless accessible. Among the resources found, it was possible to obtain a listing of users registered in the application.

Impact

Exposure of directories not intended for public access can reveal sensitive information about the application's structure, configuration files, and user data, providing support for further attacks (e.g., user enumeration for brute-force attacks).

Recommendation

- Restrict access to directories and files that should not be publicly accessible (via web server configuration);
- Remove or protect with authentication any user-listing endpoints not intended for public use;
- Conduct periodic attack surface scans to identify unintended exposures.

Evidence

```
.hta (Status: 403) [Size: 276]
.htaccess (Status: 403) [Size: 276]
.htpasswd (Status: 403) [Size: 276]
admin (Status: 301) [Size: 310] [→ http://10.0.27.160/admin/]
api (Status: 301) [Size: 308] [→ http://10.0.27.160/api/]
assets (Status: 301) [Size: 311] [→ http://10.0.27.160/assets/]
config (Status: 301) [Size: 311] [→ http://10.0.27.160/config/]
includes (Status: 301) [Size: 313] [→ http://10.0.27.160/includes/]
index.php (Status: 200) [Size: 6179]
server-status (Status: 403) [Size: 276]
test (Status: 301) [Size: 309] [→ http://10.0.27.160/test/]
uploads (Status: 301) [Size: 312] [→ http://10.0.27.160/uploads/]
```

5.9. Missing Clickjacking Protection (Iframe)

SEVERITY: MEDIUM

Description

It was found that the application does not implement adequate security headers (such as X-Frame-Options or Content-Security-Policy: frame-ancestors), allowing its pages to be loaded within an <iframe> element on a third-party domain.

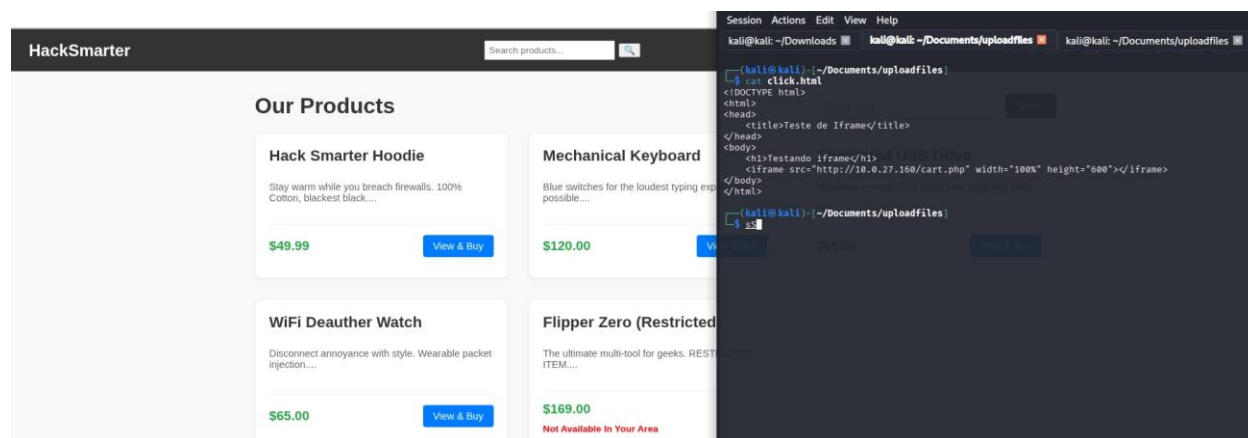
Impact

The absence of this protection enables Clickjacking attacks, in which an attacker can trick the user into unknowingly interacting with elements of the legitimate application overlaid by a malicious page, performing unauthorized actions on the victim's behalf.

Recommendation

- Implement the X-Frame-Options HTTP header with a value of DENY or SAMEORIGIN;
- Complement this with the frame-ancestors directive of the Content-Security-Policy (CSP);
- Add client-side frame-busting techniques as an additional layer (defense in depth).

Evidence



5.10. Improper Session Management (Cookie Not Invalidated)

SEVERITY: HIGH

Description

It was identified that, after logging out of the application and logging back in, the session cookie remains the same, indicating that the previous session is not properly invalidated on the server at logout.

Impact

This flaw represents a session management issue that can be exploited in session hijacking scenarios: if a session cookie is compromised (e.g., via XSS, an insecure network, or a shared device), it will remain valid even after the legitimate user logs out, allowing an attacker to continue improperly using the session.

Recommendation

- Invalidate the session on the server side immediately after logout;
- Generate a new session identifier on every new login (session fixation prevention);
- Set an appropriate expiration time for inactive sessions;
- Use cookies with the HttpOnly, Secure, and SameSite flags properly configured.

Evidence

```
de5507c8297bbfb35f0ba8abf99632ac
de5507c8297bbfb35f0ba8abf99632ac
```

6. Conclusion

During the execution of this penetration test, multiple vulnerabilities of critical, high, and medium severity were identified, with particular emphasis on the SQL Injection flaw, which allowed the extraction of user credentials and represents the highest risk identified in the assessed environment.

It is recommended that remediation be prioritized according to the risk classification presented in section 4, starting with vulnerabilities classified as Critical and High, followed by the implementation of an ongoing security testing and code review process, in order to reduce the application's attack surface and prevent the reintroduction of similar flaws.